



# PRACTICA MPI

Cristian Daranuta Zop y Augustin Dragos Belu

## Contenido

|                                       |           |
|---------------------------------------|-----------|
| <b>Implementación</b>                 | <b>2</b>  |
| <b>Diseño</b>                         | <b>4</b>  |
| <b>Análisis de resultados</b>         | <b>6</b>  |
| <b>Problemas que hemos encontrado</b> | <b>9</b>  |
| <b>Conclusión</b>                     | <b>10</b> |

## Implementación

---

Esta implementación resuelve el problema del viajante, pero usando MPI, dentro de el hay el uso del algoritmo Branch and Bound. La modelización del problema, es decir el núcleo, esta en la biblioteca de libstp donde queda definido la estructura del nodo, lo cual cada nodo tiene:

- Id: identificador del nodo.
- ci : cota inferior del subproblema.
- incl : arcos incluidos en la solución parcial.
- orig\_excl : ciudad origen de los arcos excluidos.
- dest\_excl : destinos excluidos para ese origen.

La matriz inicial es leída desde un fichero, a partir de eso cada nodo reconstruye su propia matriz de trabajo, aplica los arcos incluidos y excluidos, elimina ciclos, reduce la matriz y calcula la cota inferior.

Nuestra versión sigue un diseño de maestro-trabajadores:

1. El proceso 0 actuara como maestro.
2. Los siguientes procesos 1...N-1 actuarán como trabajadores.
3. La matriz es leída por el maestro.
4. Después toda la matriz es distribuida a todos los procesos mediante el `MPI_Bcast`.
5. Lo siguiente el maestro genera una bolsa inicial de subproblemas.
6. Así conseguimos que cada vez que un trabajador se queda con su pila de trabajo vacía va a pedir trabajo.
7. El maestro va enviar dos respuestas: un nodo pendiente o una señal de finalización.
8. Cuando uno de los trabajadores llega una solución óptima, envía la solución al maestro.
9. El maestro actualiza el ranking de las M mejores soluciones y, si cambia el umbral de poda, comunica la nueva cota al resto de procesos.
10. Cada trabajador poda su pila cada vez que recibe una cota mejor.
11. Al finalizar, el maestro recoge el total de nodos explorados de todos los procesos mediante `MPI_Reduce`.

Para poder enviar nodos entre los procesos se implementó una serialización manual con la función `serializarNodo` y `deserializarNodo`, fue necesario el uso porque el nodo contiene punteros dinámicos y eso hace que no se pueda enviar como una estructura contigua con memoria en MPI.

También tenemos definiciones en las etiquetas MPI para poder distinguir los diferentes mensajes:

`TAG_PIDE_TRAB` petición de trabajo

`TAG_TRABAJO` envío de un nodo

`TAG_FIN` finalización

`TAG_NUEVA_CS` nueva cota superior global

`TAG_SOLUCION` solución encontrada

## Diseño

---

Nuestro diseño fue paralelizar el árbol de búsqueda y no la matriz de distancias, así cada proceso explora cada rama/parte distinta del árbol.

El maestro combina el rol de coordinador con el de participante activo en el B&B. Tras generar una bolsa inicial de subproblemas con la fórmula:

```
int objetivo_trabajo = (num_procs - 1) * 8;
```

De esta forma el maestro entra en un bucle principal con tres fases por iteración:

1. Servir todas las peticiones pendientes sin bloquear mediante MPI\_Iprobe, dando prioridad siempre a los workers;
2. Si la bolsa tiene nodos, computar un lote de hasta LOTE\_MAESTRO = 64 nodos aprovechando los ciclos sobrantes, tras lo cual reactiva los workers ociosos con el trabajo nuevo generado;
3. Si la bolsa está vacía, terminar si todos los workers están ociosos, o bloquearse en MPI\_Probe cediendo el núcleo hasta recibir el siguiente mensaje. Este diseño evita el busy-polling del maestro y aprovecha el núcleo que en un esquema maestro-trabajador puro quedaría ocioso.

Como hemos dicho en el anterior punto cada trabajador tiene su propia pila de nodos, seguirá ramificando mientras tenga nodos pendientes.

La recepción de la mejor solución se realiza de forma asíncrona de los trabajadores. Para ello se usa MPI\_Iprobe con tag TAG\_NUEVA\_CS, sin mantener una petición persistente. Así podemos conseguir que un trabajador pueda seguir calculando y de forma periódica compruebe si el maestro ha enviado una nueva cota superior. Si al llegar el momento, recibe dicha cota mejor, elimina la pila de nodos cuya cota inferior no mejorará la solución actual.

Un aspecto crítico del diseño es el umbral de poda para las M mejores soluciones. El umbral correcto es el coste de la M-ésima mejor solución encontrada (costes[M-1]), no el de la mejor (costes[0]). Usar costes[0] provocaría poda excesiva, descartando candidatos válidos para los puestos 2º y 3º. Por el mismo motivo, los workers no actualizan su cota local al encontrar una solución, ya que el maestro es el único que conoce el ranking global y calcula el umbral correcto.

Este diseño nos permite distribuir el trabajo de forma eficiente, reducir las exploraciones innecesarias mediante la poda global, mantener la lógica limpia y clara de maestro-trabajador y conservar el algoritmo Branch and Bound.

## Análisis de resultados

A continuación, analizaremos los tiempos obtenidos en diferentes ejecuciones del código. Y de la misma forma, analizarlo contra las versiones secuenciales. A partir de estos tiempos se han calculado el speed-up y la eficiencia para ver si realmente merece la pena paralelizar el problema y hasta qué punto se aprovechan los procesos MPI.

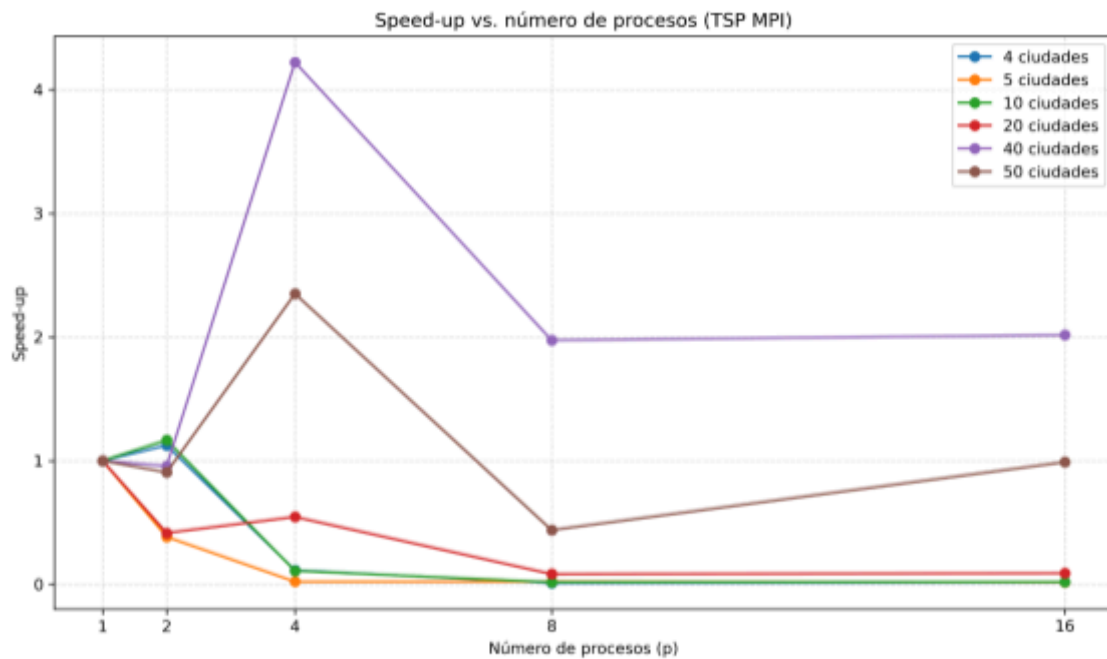


Figura 1: Speed-up en función del número de procesos

Como se puede ver en la gráfica de speed-up, en los tamaños pequeños del problema el paralelismo no aporta una mejora clara. Esto pasa porque el tiempo de cálculo es muy reducido y el coste de comunicación entre procesos acaba siendo más importante que el propio trabajo útil. Por eso, para ciudades pequeñas, la versión paralela muy parecida a los resultados de la ejecución secuencial.

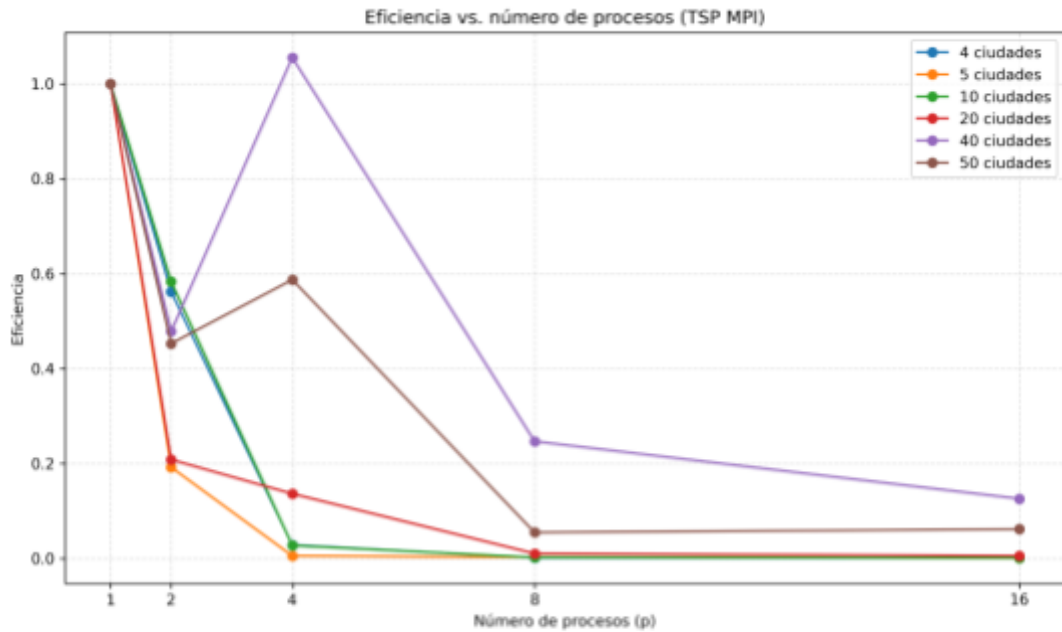


Figura 2: Eficiencia en función del número de procesos

Ahora, si nos ponemos a hablar de eficiencia en cuanto el tamaño aumenta, el comportamiento final es bastante diferente. Como podemos ver, por ejemplo, en trazas de 40-50 ciudades, tenemos una mejora muy interesante, sobre todo si trabajamos con 4 procesadores. Gracias a esto, podemos ver que el trabajo ya es suficiente como para compensar el coste de coordinación entre maestro y trabajadores, ya que el enfoque del algoritmo *Branch and Bound* empieza a sacar un buen partido al paralelismo.

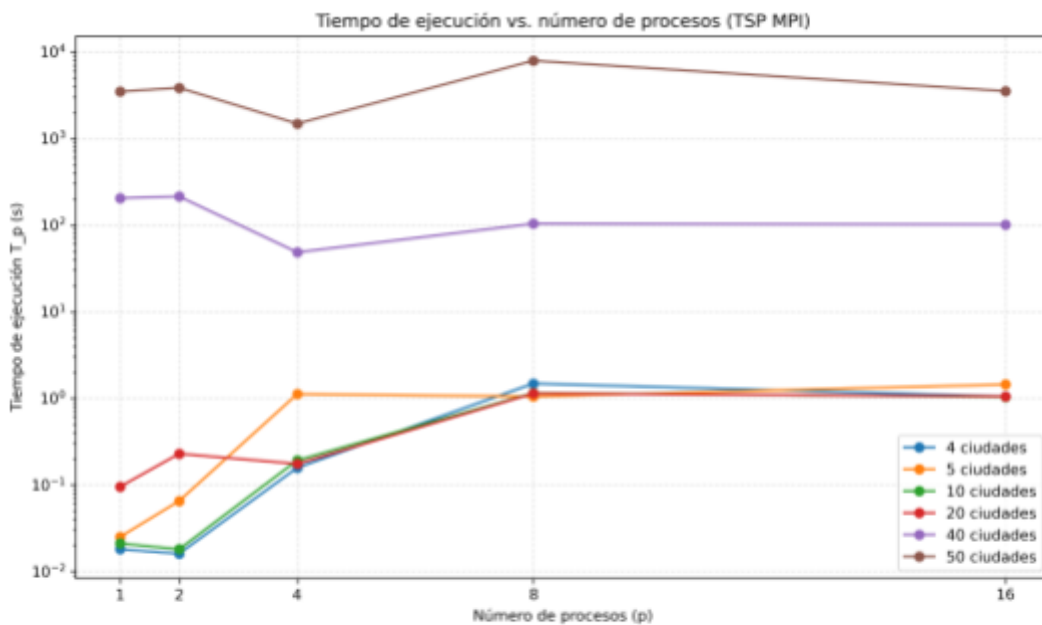


Figura 3: Tiempo de ejecución en función del número de procesos



En esta última gráfica, analizamos el tiempo de ejecución en función del número de proceso. Aquí se aprecia que el rendimiento no mejora de forma lineal al aumentar  $p$ . Esto se debe a que hay puntos que, por alguna razón de randomización, pueden afectar al balanceo de trabajo, ya que el problema que tiene el algoritmo de *Branch and Bound*, una poda temprana, puede hacer que, a lo largo del cálculo, la búsqueda se pueda descompensar de manera no uniforme entre los otros trabajadores. Esto significa que hay ramas que cuestan mucho más que otras, así que algunos procesos terminan antes y otros siguen trabajando bastante más tiempo. Ese desbalance de carga se nota sobre todo en las instancias grandes y con muchos procesos.

La gráfica de eficiencia también refleja esa misma idea. Aunque en algunos casos el speed-up mejora, la eficiencia baja cuando aumentamos mucho el número de procesos, lo que significa que el uso de los recursos no es perfecto. En otras palabras, añadir más procesos no siempre implica una mejora proporcional, porque parte del tiempo se pierde en comunicación, sincronización y reparto de trabajo.

En resumen, los resultados confirman que la versión paralela funciona correctamente y da la misma solución óptima que la secuencial, pero el rendimiento depende mucho del tamaño del problema. Para instancias pequeñas, el tiempo de paralelización no compensa, pero para instancias grandes, si podemos ver una mejora significativa, aunque con un número moderado de procesos suele obtenerse el mejor equilibrio entre tiempo y eficiencia.

Cabe recalcar, que este estudio, se realizó con la versión anterior a la entrega. Debido a que el cluster universitario no estaba accesible, por lo que no se pudo hacer el estudio de forma correcta

## Problemas que hemos encontrado

---

El principal problema que nos encontramos fue la comunicación de los nodos entre procesos ya que al contener memoria dinámica no era seguro enviarla directamente con MPI. La solución de este problema fue la serialización manual, copiando campo a campo el contenido del nodo a buffer plano de bytes.

El otro problema fue el reparto equilibrado del trabajo, dada la naturaleza del algoritmo no todos los nodos tendrán el mismo trabajo y tampoco el mismo coste computacional. Por lo tanto, para mitigarlo el maestro generara la bolsa inicial de subproblemas.

También hubo el problema de mantener la cota superior actualizada. El maestro debe distinguir entre los trabajadores que piden trabajo temporalmente y la situación de cuando no hay mas trabajos disponibles. Para mitigar se controla número de trabajadores activos y, cuando no queda trabajo pendiente, se envían mensajes de finalización a todos los procesos.

También se detectó un error en la función `cotaPoda`, que retornaba el coste de la mejor solución en lugar del de la M-ésima, causando que el ranking de las 2ª y 3ª mejores soluciones fuera incorrecto. La corrección consistió en retornar `costes[M-1]` y usar `INFINITO` como umbral mientras no se hayan encontrado M soluciones.

Por último, en una versión intermedia el maestro realizaba busy-polling con `MPI_Iprobe` cuando su bolsa estaba vacía, saturando el núcleo al 100% y robando CPU a los workers. La solución fue bloquearse en `MPI_Probe` en esa situación, cediendo el núcleo.

## Conclusión

---

Como hemos visto, en este trabajo se resolvió el problema del viajante de forma paralela, usando estructuras MPI y con un algoritmo de búsqueda *Branch and Bound*. Para ello, se implementó una estructura de *Master-Worker*, repartiendo así los subproblemas entre los procesos y usando comunicaciones asíncronas para ir actualizando la mejor solución encontrada.

Los resultados analizados muestran que el programa paralelo funciona de forma correcta, obteniendo la misma solución que la versión secuencial. También podemos ver que, para problemas pequeños, el paralelismo no es eficiente, ya que casi no aporta mejora, pero en cuanto el tamaño del problema aumenta, sí se nota la mejora. Aunque el rendimiento depende mucho también de la aleatorización del reparto del trabajo entre procesos.